



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究
A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究**
**A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

摘要

關鍵詞：大型語言模型代理人、API 任務自動化、案例式學習、動態修復

大型語言模型 (Large Language Model, LLM) 具備自然語言理解、推理與內容生成能力；當其作為代理人 (agent) 的決策核心時，可根據使用者目標進行任務規劃與工具選擇，並透過應用程式介面 (Application Programming Interface, API) 與外部系統互動。然而，在多步驟任務中，任一步驟的 API 選擇、參數綁定、資料處理或相依關係發生錯誤，都可能使後續執行偏離原始目標，因此如何利用既有成功案例輔助規劃，並針對局部失敗進行動態修復，是提升 API 任務代理人可靠度的重要議題。本研究以 API 知識圖譜代理人 (API Knowledge Graph Agent, APIKGAgent) 為基礎，提出案例式修復 API 知識圖譜代理人 (Case-Based Repair APIKGAgent, CBR-APIKGAgent)，首先從訓練資料中的成功任務萃取可跨任務重複使用的原子化經驗 (atomic experience)，並透過語意檢索提供給初始規劃、重新規劃與修復階段參考；其次設計局部修復流程，根據執行錯誤、相依步驟資料與 API 呼叫證據搜尋可能的修復 API，並以插入、取代或刪除步驟的方式調整局部計畫，當錯誤涉及原始計畫結構時則回退至重新規劃。為評估本研究方法，實驗採用 AppWorld 模擬環境，其涵蓋 9 種應用服務與 457 支 API，可呈現跨應用程式的現實多步驟任務；評估以任務是否完成並使環境產生預期狀態變化為準，並透過整體效能比較、逐步消融實驗與失敗案例分析，觀察 local repair 與 atomic experience 對任務執行的影響。研究結果顯示，在 AppWorld test_normal 資料集上，本研究取得完整任務成功率 X%；此外，局部修復對有明確錯誤訊號的情境有效，但面對較複雜的任務結構或外部狀態限制時，僅靠局部修復與經驗提示仍可能不足以穩定恢復。

ABSTRACT

Keyword: LLM agents, API task automation, case-based learning, dynamic plan repair

Large language models (LLMs) have recently been widely adopted in agent systems, where they can plan tasks, select tools, and invoke APIs according to user goals to complete real-world cross-application tasks. In multi-step API tasks, however, an error in API selection, parameter binding, data processing, or dependency handling at any step may cause the subsequent execution to deviate from the original objective. Traditional approaches usually regenerate the entire plan after an error, which may increase execution cost and disrupt the state and data flow accumulated from completed steps. Therefore, incorporating case-based guidance and local repair mechanisms into an existing planning framework is an important issue for improving the reliability of API task agents. Based on the API Knowledge Graph Agent (APIKGAgent), this study proposes the Case-Based Repair APIKGAgent (CBR-APIKGAgent). The proposed system first extracts reusable atomic experiences from successful tasks in the training data and provides them as references for initial planning, replanning, and repair through semantic retrieval. It then introduces a local repair process that searches for possible repair APIs according to execution errors, dependency information from related steps, and API call evidence, and adjusts the local plan through insertion, replacement, or deletion of steps; when an error affects the structure of the original plan, the system falls back to replanning. Experiments were conducted in the AppWorld simulation environment, which contains 9 application services and 457 APIs and reflects realistic multi-step cross-application tasks. Performance was evaluated based on task completion and the expected state changes in the environment, and the effects of local repair and atomic experience were examined through overall performance comparison, stepwise ablation studies, and failure-case analysis. On the AppWorld test_normal dataset we observe an overall task success rate of X%; local repair helps when errors have explicit signals but may be insufficient for failures caused by complex task structures or external state constraints.

誌謝

回顧這段碩士求學的歷程，從最初的摸索到最終完成論文，其中經歷了無數嘗試與修正。能夠走到這一步，並非一人之力，而是來自許多人的陪伴與支持。

首先，誠摯感謝指導教授劉建宏老師。在研究過程中，老師總能在關鍵時刻點出問題核心，引導我重新思考方向。無論是研究上的討論，或是面對困難時的提醒，都讓我逐漸建立起更成熟的思考方式，也學會如何更嚴謹地面對學術問題。同樣感謝梁文耀教授的指導與建議。您在討論中提出的觀點，往往讓我跳脫原有的框架，使研究內容得以更加周全，也讓我對自己的研究有更深一層的理解。

在實驗室的日子裡，感謝珮倫、郁喬、冠穎、以謙等夥伴的陪伴。無論是討論問題時的腦力激盪，或是日常相處中的交流與玩笑，都讓這段研究生活不再只是壓力與進度，而多了許多值得回味的片段。此外，也想感謝一路上關心與支持我的朋友們，在我感到疲憊或迷惘時給予鼓勵，讓我能重新調整步伐，繼續前進。

最後，將這份成果獻給我的家人。謝謝你們長久以來的理解與支持，讓我能專心投入學業，在面對挑戰時依然保有前進的力量。這段旅程的結束，同時也是另一段路的開始。謹以此文，致上我最真摯的感謝。

目錄

摘要	i
ABSTRACT	ii
誌謝	iii
目錄	iv
圖目錄	viii
表目錄	ix
第一章 緒論	1
1.1 研究動機	1
1.2 研究目的	2
1.3 論文組織架構	3
第二章 相關技術與文獻探討	4
2.1 大型語言模型代理人	4
2.1.1 大型語言模型代理人概述	4
2.1.2 工具使用與任務規劃	4
2.1.3 大型語言模型代理人相關研究	5
2.2 API 任務自動化	5
2.2.1 API 任務特性	6
2.2.2 API 任務自動化相關研究	6
2.2.3 AppWorld 基準環境	6
2.3 案例式學習與經驗重用	7
2.3.1 案例式推理	7
2.3.2 經驗重用相關研究	8
2.3.3 語意檢索方法	8
2.4 動態計畫修復與自我修正	9
2.4.1 重新規劃機制	9
2.4.2 自我修正機制	10
2.4.3 動態計畫修復相關研究	10

2.5	APIKGAgent	11
2.5.1	APIKGAgent 架構概述	11
2.5.2	APIKGAgent 之研究缺口	12
第三章	CBR-APIKGAgent 系統架構與方法	14
3.1	CBR-APIKGAgent 整體架構	14
3.1.1	整體架構與主要元件	14
3.1.2	與 APIKGAgent 之差異	14
3.1.3	任務執行與錯誤修復流程	14
3.2	原子化經驗重用機制	14
3.2.1	經驗來源與使用範圍	14
3.2.2	原子化經驗定義	14
3.2.3	原子化經驗表示方式	14
3.2.4	經驗檢索流程	14
3.2.5	經驗於規劃與修復流程中的應用	14
3.3	動態局部修復機制	15
3.3.1	局部修復觸發條件	15
3.3.2	錯誤資訊與相依資料蒐集	15
3.3.3	修復 API 搜尋	15
3.3.4	局部計畫修復策略	15
3.3.5	局部修復與重新規劃的切換條件	15
3.4	CBR-APIKGAgent 整合運作流程	15
第四章	系統實作	16
4.1	開發環境與工具	16
4.1.1	LangGraph	16
4.1.2	Neo4j	16
4.1.3	大型語言模型與嵌入模型	16
4.1.4	其他開發環境	16
4.2	APIKGAgent 基礎實作與修改位置	16
4.2.1	主控制流程	16

4.2.2	API 搜尋流程	16
4.2.3	任務執行與重新規劃流程	16
4.2.4	CBR-APIKGAgent 修改位置	16
4.3	原子化經驗建立與檢索實作	17
4.3.1	訓練資料成功任務軌跡萃取	17
4.3.2	原子化經驗資料格式	17
4.3.3	經驗向量化與語意檢索	17
4.3.4	Planner 經驗注入	17
4.3.5	Retry Planner 經驗注入	17
4.3.6	Rerollout 經驗注入	17
4.4	Rerollout 動態局部修復實作	17
4.4.1	修復觸發與錯誤證據整理	17
4.4.2	相依步驟資料取得	17
4.4.3	修復 API 搜尋流程	17
4.4.4	插入、替換與刪除策略實作	17
4.4.5	局部修復失敗與重新規劃回退	17
第五章	實驗與結果分析	18
5.1	研究問題	18
5.1.1	研究問題一	18
5.1.2	研究問題二	18
5.1.3	研究問題三	18
5.2	實驗設置	18
5.2.1	AppWorld 資料集與任務範圍	18
5.2.2	評估指標	18
5.2.3	實驗環境與模型設定	18
5.2.4	比較方法	19
5.3	整體效能比較	19
5.4	消融實驗	19
5.4.1	實驗組別	19

5.4.2	原子化經驗之影響	20
5.4.3	動態局部修復之影響	20
5.4.4	兩項機制之綜合影響	20
5.5	動態局部修復分析	20
5.5.1	修復觸發與執行情形	20
5.5.2	成功修復案例	20
5.5.3	修復失敗案例	20
5.5.4	錯誤類型分析	20
5.5.5	修復策略分析	20
5.6	討論	20
5.6.1	原子化經驗的影響與適用情境	20
5.6.2	動態局部修復的影響與適用情境	20
5.6.3	系統限制與能力邊界	20
第六章	結論與未來研究	21
6.1	結論	21
6.1.1	研究成果總結	21
6.1.2	研究貢獻	21
6.1.3	研究限制	21
6.2	未來研究	21
6.2.1	修復經驗累積機制	21
6.2.2	多輪局部修復策略	21
6.2.3	約束感知修復機制	21
6.2.4	跨基準資料集驗證	21
參考文獻		22
附錄		24
6.3	Prompts	24
6.4	程式碼	24

圖目錄

2.1	APIKGAgent 系統架構圖（資料來源：[2]）	12
-----	----------------------------	----



表目錄



第一章 緒論

1.1 研究動機

近年來，大型語言模型（Large Language Model, LLM）已逐漸由傳統的文字生成與問答系統，發展為能夠自主規劃、呼叫工具並操作外部系統的智慧代理人（LLM-based Agent）[1]。透過應用程式介面（Application Programming Interface, API），代理人得以執行資料查詢、訊息傳送、檔案管理、行事曆管理及交易處理等實際工作，使大型語言模型由單純提供資訊的輔助工具，進一步成為能與數位環境互動並完成使用者目標的自動化系統。因此，如何提升代理人在真實環境中的任務完成能力與執行可靠性，已成為大型語言模型研究的重要方向之一。

然而，真實世界中的 API 任務通常涉及多個相互依賴的執行步驟，而非單次 API 呼叫即可完成。代理人除了需要理解使用者需求並選擇適當的 API 外，還必須進行多步驟規劃、維護步驟間的資料依賴關係、保存中間執行結果，並持續遵循任務條件與限制。當任務規模增加或操作流程變得複雜時，任何一個步驟的錯誤都可能影響後續決策，使錯誤沿著執行流程持續傳播，最終導致整體任務失敗。因此，如何降低錯誤傳播對任務執行結果的影響，已成為提升多步驟 API 代理人可靠性的重要挑戰。

本研究延續 APIKGAgent [2] 之研究架構進行探討。APIKGAgent 透過 API 知識圖譜、語意檢索及任務規劃機制協助代理人完成多步驟 API 任務，並具備基本的重新規劃能力。然而，在複雜任務執行過程中仍可觀察到錯誤傳播與局部失敗難以修復等現象。此外，既有架構尚未系統化利用由訓練資料成功任務軌跡所萃取的策略經驗。此現象顯示，除了提升即時規劃能力之外，如何利用由訓練資料成功任務軌跡所萃取的經驗知識輔助決策，以及如何針對局部錯誤進行修復，亦可能是影響整體任務完成能力的重要因素。

綜合上述觀察，目前多步驟 API 任務代理人仍面臨兩項關鍵挑戰：

- **全域經驗利用不足：**既有系統缺乏對訓練資料中成功任務經驗的系統化利用。代理人在不同任務中經常面臨相似的規劃與資料依賴問題，但既有系統尚未將成功任務軌跡所包含的策略知識整理為可檢索的經驗，使代理人在面對相似任務結構時，缺乏可供規劃與修復參考的依據。
- **局部錯誤修復能力不足：**當任務執行過程中發生錯誤時，現有系統多仰賴重新產生整體計畫進行修正。然而，許多錯誤並非來自整體計畫設計不良，而可能僅發生於部分步驟。若直接重新規劃整體任務，不僅增加額外成本，也可能破壞原本正確的執行流程，甚至導致新的錯誤產生。

基於上述觀察，本研究認為提升多步驟 API 代理人可靠性的關鍵，不僅在於大型語言模型本身的推理能力，也在於如何利用由訓練資料成功任務軌跡所萃取的策略知識，並針對執行期間的局部錯誤進行適當修復。因此，本研究探討如何從訓練資料中

的成功任務軌跡萃取細粒度經驗，建立離線全域經驗庫，並透過語意檢索將相關經驗應用於初始規劃、重新規劃與局部修復流程；同時研究如何根據執行結果、步驟相依資料與錯誤證據進行動態局部修復。藉由經驗重用與動態修復機制的結合，本研究將探討全域經驗是否能為相似任務結構提供有效的規劃與修復參考，以及局部修復是否有助於代理人由執行失敗中恢復，進而評估兩項機制對多步驟 API 任務完成能力與錯誤修復表現的影響。

1.2 研究目的

本研究針對大型語言模型代理人在多步驟 API 任務中所面臨之錯誤傳播、訓練資料中成功任務策略尚未被系統化利用，以及局部失敗不易修復等問題，提出案例式修復 API 知識圖譜代理人（Case-Based Repair APIKGAgent, CBR-APIKGAgent）。其中，CBR 指 Case-Based Repair，表示本研究承接案例式學習與案例式推理中的檢索與重用概念，並將其應用於 API 任務的規劃與修復流程。本研究延續 APIKGAgent [2] 之架構進行探討，並在其 API 知識圖譜、API 檢索、任務規劃與步驟式執行流程上，加入經驗重用與動態局部修復機制，以提升代理人在複雜 API 任務中的自我修正能力。

具體而言，本研究之目的如下：

1. 建立訓練資料成功經驗的萃取與重用機制

本研究從訓練資料中的成功任務軌跡萃取可跨任務重用的原子化經驗（Atomic Experience），將適用情境、規劃原則、必要中間概念與常見陷阱轉換為結構化經驗知識。透過語意檢索機制，系統可在初始規劃、重新規劃與局部修復階段取得與當前情境相關的經驗，以提供代理人規劃與修復時的策略參考，並探討此機制是否有助於減少已知失敗模式在相似任務中出現。

2. 建立動態局部修復機制以處理執行失敗

本研究探討代理人在執行失敗時，如何根據失敗步驟、錯誤訊息、相依資料與 API 執行結果判斷修復方向。當錯誤可由局部調整解決時，系統透過插入、替換或刪除步驟修正局部計畫；當錯誤涉及整體任務理解或計畫結構時，則回退至重新規劃，以減少不必要的整體重新規劃，並降低重新規劃對既有正確步驟與資料流程造成干擾的可能性。

3. 建構具備自我修正能力之多步驟 API 任務代理人

本研究整合經驗重用與動態局部修復機制，建構 CBR-APIKGAgent，使代理人能於規劃、執行、錯誤偵測、經驗檢索、局部修復與重新規劃之間形成完整的自我修正流程。藉由此架構，本研究期望提升代理人在多步驟 API 任務中的任務完成能力與失敗恢復能力。

4. 探討經驗重用與動態修復機制之影響

本研究進一步探討經驗重用與動態修復機制對代理人自我修正能力之影響，並分析其於不同錯誤情境下的適用性與限制。透過 AppWorld [3] 多應用程式 API 任務環境進行評估，本研究期望釐清上述機制對任務完成率與錯誤修復表現的實際影響。

綜合而言，本研究旨在驗證：結合由訓練資料成功任務軌跡所萃取之全域經驗與動態局部計畫修復，是否能強化大型語言模型代理人在多步驟 API 任務中的自我修正能力，並進一步釐清兩項機制於不同錯誤情境下的效益與能力邊界。

1.3 論文組織架構

本論文共分為六章，各章內容概述如下：

- **第一章緒論：**說明研究背景、研究動機與研究目的，並概述本論文的整體章節安排。
- **第二章相關技術與文獻探討：**整理大型語言模型代理人、API 任務自動化、案例式學習與經驗重用、動態計畫修復，以及 APIKGAgent 等相關技術與研究。
- **第三章 CBR-APIKGAgent 系統架構與方法：**介紹 CBR-APIKGAgent 的整體架構、原子化經驗重用機制、動態局部修復機制，以及系統的整合運作流程。
- **第四章系統實作：**說明開發環境、APIKGAgent 基礎實作與修改位置，以及原子化經驗建立、語意檢索、提示注入與 Rerollout 動態局部修復等實作細節。
- **第五章實驗與結果分析：**說明研究問題、實驗設置與評估指標，並透過整體效能比較、消融實驗及動態局部修復分析，評估本研究方法對任務完成率與錯誤修復表現的影響。
- **第六章結論與未來研究：**總結研究成果、研究貢獻與限制，並提出修復經驗累積、多輪局部修復、約束感知修復及跨基準驗證等未來研究方向。

第二章 相關技術與文獻探討

2.1 大型語言模型代理人

大型語言模型代理人 (Large Language Model Agent, LLM Agent) 係指以大型語言模型 (Large Language Model, LLM) 作為核心決策元件，並結合外部工具、記憶機制與環境互動能力之智慧代理系統。相較於傳統大型語言模型僅能根據輸入文字產生回應，LLM Agent 能夠根據任務目標進行推理、規劃、工具調用以及結果反思，使其具備執行複雜任務的能力 [1]。

近年來，隨著 GPT-4、Claude 等大型語言模型能力提升，研究者開始將其應用於任務導向情境，透過整合工具使用 (Tool Use)、任務規劃 (Task Planning)、記憶管理 (Memory Management) 與行動執行 (Action Execution) 等能力，使代理人能夠完成資訊搜尋、資料分析、程式生成以及多步驟任務自動化等工作。因此，LLM Agent 已逐漸成為大型語言模型應用的重要發展方向之一。

2.1.1 大型語言模型代理人概述

傳統大型語言模型主要透過文字輸入與輸出進行互動，其知識來源侷限於訓練資料與提示內容，因此難以取得即時資訊或操作外部系統。為克服此限制，LLM Agent 將大型語言模型視為決策核心，並透過外部工具擴展其能力，使其能夠感知環境、進行推理並執行實際操作。

Wang 等人 [1] 將 LLM Agent 描述為由感知 (Perception)、推理 (Reasoning) 與行動 (Action) 三個主要能力所組成。其中，感知能力負責理解使用者需求與環境資訊；推理能力負責任務分析、規劃與決策；而行動能力則透過工具或外部系統完成實際操作。此架構使代理人能夠將複雜任務拆解為多個子任務，並根據執行結果持續調整後續行為。

此外，部分研究進一步加入記憶 (Memory) 機制，使代理人能夠保存過往任務資訊與執行結果，提升長期任務執行能力與決策品質 [4]。透過結合記憶、規劃與工具使用能力，LLM Agent 已逐漸由單純的對話系統演變為能夠自主完成任務的智慧代理人。

2.1.2 工具使用與任務規劃

由於大型語言模型本身無法直接存取外部系統，因此工具使用 (Tool Use) 已成為現代 LLM Agent 的核心能力之一。工具使用係指代理人透過 API、函式呼叫 (Function Calling) 或其他外部工具介面執行特定操作，例如資料查詢、網頁搜尋、資料庫存取或檔案處理等。

Yao 等人提出 ReAct (Reasoning and Acting) 方法，將推理 (Reasoning) 與行動 (Acting) 整合於同一流程中，使大型語言模型能夠交替進行思考與工具操作 [5]。

ReAct 透過推理過程產生下一步行動，並根據工具執行結果更新後續決策，有效提升代理人在複雜任務中的問題解決能力。

然而，當任務規模增加時，僅依賴逐步推理可能導致規劃不完整或錯誤累積。因此，研究者開始探討任務規劃（Task Planning）機制，使代理人能夠先建立整體任務計畫，再依序執行各項步驟。Wang 等人提出 Plan-and-Solve 方法，透過先規劃後執行（Plan-then-Solve）的策略，降低大型語言模型直接推理時可能產生的遺漏步驟問題 [6]。

對於多步驟任務而言，規劃能力不僅涉及任務拆解，亦包含步驟間資料依賴關係管理與執行順序安排。當任務需要呼叫多個 API 或整合不同工具時，代理人必須確保前一階段產生之結果能夠正確傳遞至後續步驟，否則可能導致任務失敗。因此，工具使用與任務規劃能力已成為現代 LLM Agent 架構的重要組成部分。

2.1.3 大型語言模型代理人相關研究

隨著 LLM Agent 發展，研究者開始探討如何提升代理人在複雜任務中的決策能力、自我修正能力以及長期任務執行能力。

ReAct [5] 將推理與工具操作整合於單一流程中，使代理人能夠根據工具執行結果持續更新推理過程，為後續代理人研究奠定基礎。然而，ReAct 主要採用逐步決策方式，缺乏全域任務規劃能力，因此在長流程任務中可能產生錯誤累積問題。

為改善大型語言模型的規劃能力，Plan-and-Solve [6] 提出先規劃後執行的架構，使模型在執行前先建立完整任務計畫，以降低遺漏步驟與推理錯誤。然而，此類方法仍高度依賴初始規劃品質，當執行過程發生錯誤時，往往缺乏有效的修正機制。

在自我修正方面，Shinn 等人提出 Reflexion 架構，透過語言回饋（Verbal Feedback）機制使代理人能夠根據過往失敗經驗進行反思與修正 [7]。該方法利用執行結果作為回饋訊號，協助代理人改善後續決策品質，展現出大型語言模型具備一定程度自我修正能力的可能性。

此外，Park 等人提出 Generative Agents 架構，透過記憶儲存、檢索與反思機制，使代理人能夠根據歷史經驗調整後續行為 [4]。此研究顯示記憶與經驗重用對於提升代理人長期決策能力具有重要影響。

綜合上述研究可以發現，目前 LLM Agent 已具備工具使用、任務規劃、自我修正與記憶管理等能力。然而，當代理人面對多步驟 API 任務時，仍可能遭遇規劃錯誤、執行失敗以及錯誤修復效率不足等問題。因此，如何結合經驗重用與動態修復機制以提升代理人在複雜任務中的自我修正能力，仍是值得進一步探討的重要研究方向。

2.2 API 任務自動化

近年來，隨著大型語言模型代理人逐漸具備工具使用能力，研究者開始探討如何透過應用程式介面（Application Programming Interface, API）使代理人能夠與外部系統互動，進而完成實際任務。相較於傳統僅產生文字回應的大型語言模型，具備 API 呼

叫能力的代理人可執行資料查詢、訊息傳送、檔案管理、行事曆安排及電子商務等操作，使大型語言模型由資訊提供者轉變為任務執行者。因此，API 任務自動化已成為大型語言模型代理人研究的重要方向之一。

2.2.1 API 任務特性

API 任務係指代理人透過呼叫一個或多個 API 完成特定目標之過程。與一般問答任務不同，API 任務通常涉及外部環境狀態變化，因此代理人不僅需要產生正確答案，更必須透過實際操作使系統狀態符合使用者需求。

在簡單情況下，任務可能僅需單次 API 呼叫即可完成。然而，在真實應用情境中，多數任務通常包含多個相互依賴的步驟。例如，代理人可能需要先搜尋符合條件的使用者，再取得詳細資訊，最後執行修改或刪除操作。此類任務不僅涉及 API 選擇問題，也包含資料依賴管理、參數建構與執行順序規劃等挑戰。

此外，API 任務通常具有狀態性（Statefulness）特性。代理人在執行過程中所產生的結果將影響後續步驟的執行條件，因此必須正確保存與使用中間資料。若任一步驟發生錯誤，錯誤可能沿著後續流程持續傳播，最終導致整體任務失敗。此特性使 API 任務相較於一般文字推理任務更具挑戰性。

2.2.2 API 任務自動化相關研究

為提升大型語言模型於工具使用與 API 操作情境中的能力，研究者提出多種 API 任務自動化方法。

ToolBench [8] 建立大規模工具學習資料集，涵蓋大量真實 API 與工具操作情境，並透過工具呼叫軌跡訓練大型語言模型，使其能夠學習工具選擇與參數使用方式。此研究顯示大型語言模型可透過工具使用資料提升外部工具操作能力。

ToolLLM [9] 進一步探討大型語言模型於大規模 API 環境中的工具使用能力。該研究蒐集超過一萬六千個真實 API，並提出 API 檢索與工具使用流程，使代理人能夠從大量候選工具中選擇適當 API 完成任務。研究結果顯示，當 API 數量增加時，如何有效搜尋與選擇工具將成為影響代理人表現的重要因素。

然而，多數 API 任務研究主要聚焦於工具選擇與 API 呼叫能力，較少考量長流程任務中的規劃、錯誤修復與經驗重用問題。當任務涉及多個 API、跨應用程式操作或複雜資料依賴關係時，代理人仍可能因規劃錯誤、資料遺失或執行失敗而無法完成任務。因此，如何提升代理人在複雜多步驟 API 任務中的執行能力，仍是重要研究議題。

2.2.3 AppWorld 基準環境

為評估大型語言模型代理人於真實應用情境中的任務執行能力，Trivedi 等人提出 AppWorld 基準環境（Benchmark Environment）[3]。AppWorld 建立一個模擬多應用程式

生態系統，包含電子郵件、行事曆、購物、銀行、社群媒體等多種應用服務，並提供數百個 API 供代理人操作。

與傳統問答資料集不同，AppWorld 採用可執行環境（Executable Environment）作為評估基礎。代理人必須透過 API 呼叫實際改變環境狀態，並達成指定任務目標，才能被視為成功完成任務。因此，評估結果不僅取決於文字輸出是否正確，更取決於最終環境狀態是否符合任務需求。

此外，AppWorld 所設計之任務多為跨應用程式、多步驟任務。例如，代理人可能需要先查詢電子郵件內容，再根據資訊建立行事曆事件，或透過多個 API 完成資料搜尋與更新操作。此類任務能夠有效測試代理人的規劃能力、工具使用能力以及錯誤修復能力，因此已逐漸成為評估大型語言模型代理人的重要基準環境。

由於本研究所探討之問題涉及多步驟 API 任務中的經驗重用與動態修復機制，因此選擇 AppWorld 作為主要實驗平台，以驗證所提出方法於複雜 API 任務環境中的實際效益。

2.3 案例式學習與經驗重用

案例式學習（Case-Based Learning）係利用既有案例所包含的問題情境、處理策略與執行結果，協助系統處理新的相似問題。此類方法的核心並非直接複製過往解答，而是先判斷新舊情境的相似性，再選擇可重用的知識並依目前條件調整。對大型語言模型代理人而言，任務軌跡記錄了任務目標、規劃步驟、工具呼叫、資料依賴與執行結果，因而可作為經驗知識的重要來源。然而，完整軌跡通常包含大量僅適用於單一任務的實體、參數與中間資料，若未經整理便直接提供給代理人，可能增加提示內容並引入與當前任務無關的資訊。因此，如何將任務軌跡轉換為適當粒度的經驗，並在需要時檢索相關內容，是經驗重用機制的重要課題。

2.3.1 案例式推理

案例式推理（Case-Based Reasoning, CBR）是一種以過往問題求解經驗處理新問題的方法。Aamodt 與 Plaza 將其概括為檢索（Retrieve）、重用（Reuse）、修訂（Revise）與保留（Retain）四個階段所構成的循環 [10]。檢索階段從案例庫中找出與當前問題相似的案例；重用階段將既有案例的解法套用或調整至新問題；修訂階段根據實際結果檢查並修正解法；保留階段則將新的求解經驗整理後存回案例庫，供後續問題使用。

CBR 的案例通常由問題描述、解決方案及結果等資訊組成，而案例是否能有效重用，取決於情境表示、相似性判斷與解法調整方式。若案例表示過於具體，檢索結果可能只適用於幾乎相同的任務；若抽象程度過高，則可能失去執行時所需的條件與限制。因此，案例設計需在可重用性與情境完整性之間取得平衡。

將 CBR 概念應用於多步驟 API 任務時，可將使用者目標、任務結構與資料依賴視為問題情境，將規劃原則、必要中間資訊與操作限制視為可重用解法。由於不同任務

所使用的應用程式、實體名稱與參數值可能不同，直接複製完整成功計畫未必適用於新任務；相較之下，從成功軌跡萃取適用條件與策略原則，較能保留跨任務重用的可能性。此觀點構成本研究將成功任務軌跡整理為原子化經驗的概念基礎。

2.3.2 經驗重用相關研究

在大型語言模型代理人研究中，經驗通常以自然語言反思、記憶片段、完整任務軌跡或抽象策略等形式保存。Park 等人提出 Generative Agents，將代理人的觀察保存於記憶串流，並根據時近性、重要性與相關性檢索記憶，同時透過反思產生較高層次的概念 [4]。此研究說明外部記憶不僅可保存歷史事件，也能透過檢索與整理影響後續規劃。

Reflexion 則利用文字回饋建立代理人的反思記憶，使代理人在不更新模型參數的情況下，根據先前嘗試的結果調整後續決策 [7]。此方法顯示自然語言形式的經驗可作為代理人的決策提示，但其經驗主要來自代理人先前的試誤過程，與預先由訓練資料建立經驗庫的設定有所不同。

Zhao 等人提出 ExpeL，讓代理人從一組訓練任務的執行經驗中萃取自然語言知識，並於推論階段回想相關洞見與過往軌跡，以協助新的任務決策 [11]。此方法與 CBR 皆強調從既有任務經驗尋找可轉移知識，也說明代理人可藉由外部經驗庫取得參考，而不必修改大型語言模型的參數。

上述研究顯示，經驗重用大致可分為保存原始軌跡與萃取抽象知識兩種方向。原始軌跡保留較完整的行動與觀察資訊，但可能包含任務特定內容與較多雜訊；抽象知識較容易跨任務使用，但若缺少適用情境、必要中間資訊或常見陷阱，也可能被錯誤套用。因此，本研究從訓練資料中的成功任務證據離線萃取結構化原子化經驗，將完整任務中可跨任務重用的策略知識整理為較細粒度的經驗單元，並保留其適用情境、核心原則、必要中間概念與常見陷阱。這些經驗提供給代理人的規劃、重新規劃與局部修復流程作為策略參考，而不是直接複製來源任務的完整解題步驟。

需特別說明的是，本研究主要借用 CBR 與經驗重用研究中的檢索與重用概念，正式經驗庫由訓練資料中的成功任務軌跡離線建立；測試期間產生的修復經驗不會寫回正式檢索經驗庫，因此本研究不涉及測試期間的線上經驗累積、持續學習或完整 CBR Retain 循環。

2.3.3 語意檢索方法

經驗庫規模增加後，系統需要從大量經驗中選出與當前情境相關的內容。傳統關鍵字檢索主要依賴詞彙重疊，當任務與經驗使用不同描述方式時，可能無法找出語意相近的案例。語意檢索則將查詢與候選內容轉換為稠密向量，並在向量空間中計算相似程度。Karpukhin 等人提出 Dense Passage Retrieval，利用雙編碼器分別建立查詢與文本的稠密表示，說明稠密向量可用於找出語意相關而不僅是詞彙相同的內容 [12]。

Lewis 等人提出檢索增強生成 (Retrieval-Augmented Generation, RAG)，將預訓練生成模型視為參數化記憶，並以可檢索的稠密向量索引作為非參數化記憶，使模型能依據查詢取得外部內容後再進行生成 [13]。此概念使知識可獨立於模型參數保存與更新，也適合用於將任務經驗提供給大型語言模型代理人參考。

語意檢索常使用餘弦相似度衡量查詢向量 $\mathbf{q}$ 與經驗向量 $\mathbf{e}$ 的接近程度，其定義如下：

$$\text{sim}(\mathbf{q}, \mathbf{e}) = \frac{\mathbf{q} \cdot \mathbf{e}}{\|\mathbf{q}\| \|\mathbf{e}\|}. \quad (2.1)$$

系統可依相似度排序並取得前 k 筆經驗，再將其加入代理人的提示內容。然而，語意相似並不代表策略必然適用；若查詢內容未包含當前階段的目標、失敗資訊或資料依賴，仍可能檢索到表面相似但無法使用的經驗。此外，檢索數量過多也可能增加無關資訊，干擾模型原有判斷。因此，經驗內容的結構、查詢資訊的選擇與檢索結果的注入方式，皆會影響經驗重用的實際表現。

依據上述概念，本研究以語意檢索從訓練資料建立的原子化經驗庫中取得與當前任務或失敗情境相關的內容，並分別提供給初始規劃、重新規劃與局部修復流程。由於三個階段所面對的資訊不同，檢索時需考量任務目標、既有計畫、失敗步驟與錯誤資訊等情境，使取得的經驗能對應目前的決策需求。檢索到的內容僅作為策略參考，而非代理人必須遵循的固定規則；此機制是否有助於多步驟 API 任務的規劃與修復，仍需透過後續實驗加以評估。至於原子化經驗的欄位設計、索引建立方式與各階段的查詢及注入流程，將於第三章與第四章進一步說明。

2.4 動態計畫修復與自我修正

大型語言模型代理人在多步驟任務中通常需要先產生計畫，再依序呼叫工具或 API 完成任務。然而，在實際執行過程中，代理人可能因任務理解錯誤、API 選擇不當、參數推論錯誤、資料依賴缺失或中間結果不符合預期而導致任務失敗。此類錯誤若未能即時處理，往往會沿著後續步驟傳遞，形成錯誤傳播 (Error Propagation) 問題，使後續計畫即使本身合理，也可能因依賴錯誤資料而無法成功執行。

因此，近年大型語言模型代理人研究逐漸重視執行回饋、重新規劃與自我修正能力。相較於一次性產生完整答案或完整計畫，具備修正能力的代理人能夠根據環境回饋、工具執行結果或錯誤訊息調整後續行動。此類方法顯示，代理人不僅需要初始規劃能力，也需要在執行過程中偵測錯誤、理解失敗原因，並根據當前狀態產生新的行動策略。

2.4.1 重新規劃機制

重新規劃 (Replanning) 是多步驟代理人常見的錯誤處理方法。當代理人在執行過程中遇到錯誤或原始計畫無法繼續完成時，系統會根據目前狀態、已完成步驟、失敗

資訊與任務目標重新產生後續計畫。此機制使代理人能夠在初始計畫不完整或環境狀態發生變化時，重新調整任務執行方向。

在大型語言模型代理人中，重新規劃通常與工具執行回饋結合。ReAct (Reasoning and Acting) 將推理與行動交替進行，使模型能夠根據工具回傳結果更新後續推理與行動決策 [5]。此類方法展現了代理人根據執行觀察調整行為的能力，也為後續具備執行回饋與重新決策能力的代理人架構奠定基礎。

然而，對於多步驟 API 任務而言，完全重新規劃並非總是最適合的修復方式。當錯誤僅發生於局部步驟，例如缺少某個中間資料、API 參數錯誤，或單一步驟使用了不適合的 API 時，若直接重新產生整體計畫，可能使原本已正確完成的步驟與資料流程被改寫，進而引入新的不確定性。此外，API 任務通常具有明確的資料依賴關係，前一步驟取得的識別碼、查詢結果或中間狀態，常會影響後續 API 呼叫。因此，重新規劃雖然能提供較大範圍的修正能力，但也可能造成不必要的整體變動。

2.4.2 自我修正機制

自我修正 (Self-Correction) 是指大型語言模型或代理人根據自身輸出、外部回饋或執行結果，對原始答案、推理過程或行動計畫進行檢查與修改的能力。此類方法通常不需要更新模型參數，而是透過提示設計、回饋訊號或記憶機制，使模型在推論階段改善原始輸出。

Reflexion 提出以語言回饋作為代理人自我改進的方式，使代理人能夠根據任務失敗訊號產生反思內容，並將其作為後續嘗試的決策依據 [7]。此方法顯示，文字形式的失敗分析與經驗描述可以在不進行模型微調的情況下，輔助代理人改善後續行動。

Self-Refine 則進一步展示大型語言模型可透過自我回饋與反覆修訂流程改善初始輸出 [14]。其核心概念為先產生結果，再由模型自行分析缺失並提出修改建議，最後依據回饋重新生成內容。研究結果顯示，即使不進行額外訓練，模型仍能透過反覆修正提升輸出品質。

在工具使用與 API 任務場景中，自我修正的回饋來源不僅包含模型自身判斷，也包含 API 回傳結果、錯誤訊息、執行紀錄與中間資料狀態。這些外部訊號能提供比純文字生成任務更具體的錯誤線索。例如，API 呼叫失敗可能指出缺少必要參數，查詢結果為空可能表示前置搜尋條件不正確，而後續步驟無法執行則可能反映資料依賴尚未建立。若代理人能夠利用這些訊號分析失敗原因，便有機會針對錯誤位置進行修正，而非完全依賴重新產生整體計畫。

2.4.3 動態計畫修復相關研究

動態計畫修復 (Dynamic Plan Repair) 關注代理人在執行過程中面對錯誤、環境變化或計畫不可行時，如何根據當前狀態修正原有計畫。相較於重新產生完整計畫，計畫修復更強調保留原計畫中仍然有效的部分，並針對失敗區段進行局部調整。

近年研究開始探討如何結合驗證與修復機制提升代理人執行能力。DVR (Divide-Verify-Refine) 提出將複雜任務拆解後逐步驗證，並根據驗證結果進行修正的流程 [15]。其核心概念在於利用外部驗證訊號找出失敗原因，再透過修正程序改善後續執行結果。若與前述 Reflexion 及 Self-Refine 共同觀察，可發現這些研究雖然採用不同形式的回饋來源與修正流程，但皆支持一項共同觀點：大型語言模型代理人可利用失敗訊號、驗證結果或自我回饋，對原始推理或行動計畫進行後續修正。此類方法顯示，相較於直接重新產生完整解答，利用失敗資訊進行針對性修正有助於改善代理人的錯誤修復表現。

對 API 任務而言，動態計畫修復具有特別的重要性。API 任務中的每個步驟通常不只是單一動作，而是會產生後續步驟所需的資料，例如物件識別碼、查詢結果、時間條件、使用者資訊或狀態更新結果。因此，局部錯誤可能導致後續步驟缺少必要輸入，進而造成連鎖失敗。若代理人能在錯誤發生後辨識其影響範圍，並針對局部計畫進行插入、替換或刪除，便能在不完全推翻原計畫的情況下嘗試恢復任務執行。

綜合上述研究可以發現，重新規劃著重於重新建立任務執行流程，自我修正強調根據回饋改善決策品質，而動態計畫修復則介於兩者之間，試圖利用執行過程中的錯誤資訊修正局部計畫。此概念亦成為本研究設計動態局部修復機制的重要基礎。

2.5 APIKGAgent

APIKGAgent (API Knowledge Graph Agent) 為吳柏諭所提出之大型語言模型代理人架構，其核心概念為結合 API 知識圖譜 (API Knowledge Graph) 與大型語言模型，使代理人能夠在大量 API 環境中完成多步驟任務 [2]。相較於直接將完整 API 文件提供給大型語言模型進行推理，APIKGAgent 透過知識圖譜保存 API、參數以及參數依賴關係，並利用圖譜搜尋結果輔助任務規劃與執行，以降低大型語言模型於 API 檢索與參數推理過程中的錯誤。

2.5.1 APIKGAgent 架構概述

APIKGAgent 的整體流程可分為 API 搜尋、任務規劃、任務執行與重新規劃四個階段，其架構如圖 2.1 所示 [2]。

首先，在 API 搜尋階段，系統根據使用者任務描述擷取關鍵資訊，並透過 API 知識圖譜搜尋可能相關之 API。由於知識圖譜中保存了 API 與參數之間的依賴關係，因此系統能夠有效縮小搜尋範圍，避免大型語言模型直接面對大量 API 文件所造成的推理負擔。

完成 API 搜尋後，任務規劃模組會根據使用者需求與候選 API 集合產生任務計畫 (Task Plan)。系統將複雜任務拆解為多個步驟，並決定各步驟所需之 API 呼叫順序與資料流向，使後續執行流程能夠依照規劃逐步完成任務。

在任務執行階段，代理人依照規劃結果逐步執行 API 呼叫與資料處理流程。執行

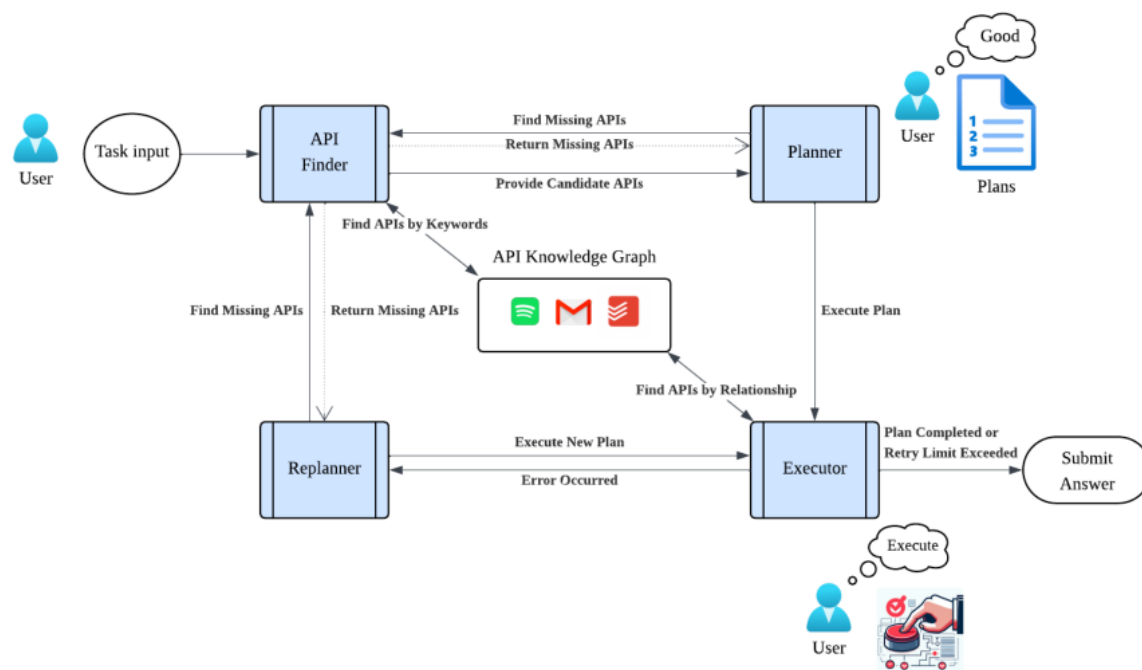


圖 2.1: APIKGAgent 系統架構圖（資料來源：[2]）

過程中所產生的中間結果會被保存，供後續步驟使用，以維持多步驟任務中的資料依賴關係。此外，系統亦會記錄各步驟之執行結果，作為後續錯誤分析與重新規劃的依據。

當任務執行失敗或無法繼續完成時，系統則啟動重新規劃（Replanning）機制。重新規劃模組會根據目前執行狀態、錯誤資訊以及已完成步驟重新產生後續計畫，使代理人能夠在部分步驟失敗的情況下持續嘗試完成任務。

透過 API 知識圖譜、任務規劃、任務執行以及重新規劃等模組的整合，APIKGAgent 建立了一套完整的多步驟 API 任務執行流程，並驗證了知識圖譜輔助 API 搜尋與規劃機制在複雜 API 任務中的可行性。

2.5.2 APIKGAgent 之研究缺口

雖然 APIKGAgent 已能透過知識圖譜改善 API 搜尋效率，並利用重新規劃機制提升多步驟任務之完成能力，但在實際任務執行過程中仍存在若干限制。

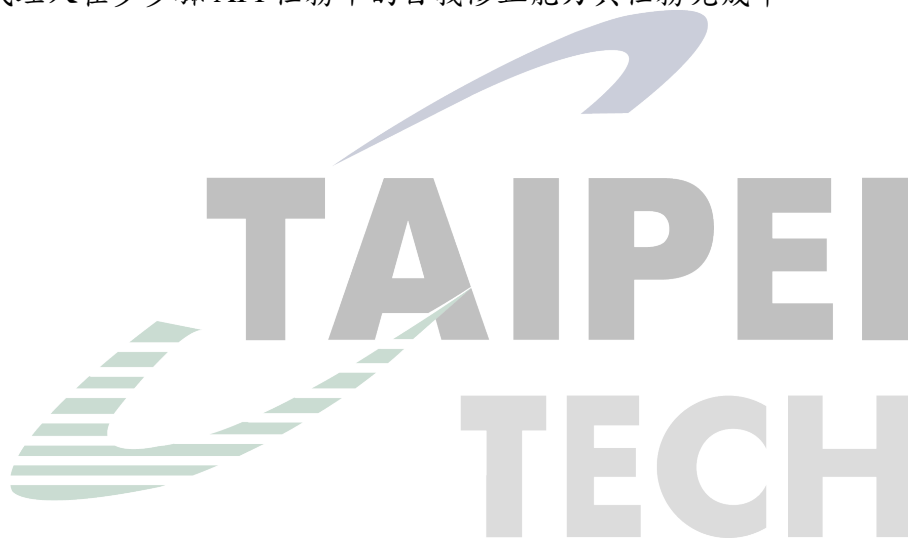
首先，當任務執行失敗時，重新規劃機制主要依賴大型語言模型根據當前執行狀態重新產生計畫。然而，本研究接續之基礎版本尚未將成功軌跡整理為可檢索、可重用的經驗庫，因此當面對相似問題時，代理人缺乏可供參考的跨任務策略知識，仍可能重複產生相同類型的規劃錯誤或執行錯誤。換言之，系統尚未系統化利用跨任務經驗重用（Experience Reuse）輔助規劃與修復。

其次，APIKGAgent 的錯誤修復主要依賴重新規劃機制處理執行失敗情況。然而，在多步驟任務中，部分錯誤可能僅發生於局部步驟，例如遺漏必要資訊蒐集步驟、錯

誤使用中間資料，或選擇不適當的 API 等情況。此類錯誤未必需要重新產生整體計畫即可修正，但現有架構缺乏針對局部錯誤進行修復的能力。

此外，重新規劃結果高度依賴大型語言模型於當下的推理能力，缺乏可供參考之歷史執行經驗與修復策略。因此，當任務結構較為複雜或錯誤原因較難辨識時，系統仍可能出現重複失敗、修復方向偏離原始任務目標，或重新規劃後仍無法有效解決問題等情況。

綜合上述分析可以發現，現有 APIKGAgent 雖已具備 API 搜尋、任務規劃、執行監控與重新規劃等能力，但在經驗重用與局部錯誤修復方面仍有進一步改善空間。因此，本研究提出案例式修復 API 知識圖譜代理人（Case-Based Repair APIKGAgent, CBR-APIKGAgent），透過原子化經驗（Atomic Experience）重用機制與動態局部修復機制，提升代理人在多步驟 API 任務中的自我修正能力與任務完成率。



第三章 CBR-APIKGAgent 系統架構與方法

3.1 CBR-APIKGAgent 整體架構

3.1.1 整體架構與主要元件

本節介紹 Planner、Executor、Retry Planner、Atomic Experience Retriever、Rerollout 與 API Knowledge Graph 等主要元件。

3.1.2 與 APIKGAgent 之差異

本節說明 CBR-APIKGAgent 新增的原子化經驗重用與動態局部修復機制。

3.1.3 任務執行與錯誤修復流程

3.2 原子化經驗重用機制

3.2.1 經驗來源與使用範圍

本研究使用的經驗來自訓練資料中的成功任務軌跡，不於測試期間進行線上累積。

3.2.2 原子化經驗定義

3.2.3 原子化經驗表示方式

3.2.4 經驗檢索流程

3.2.5 經驗於規劃與修復流程中的應用

本節說明 Atomic Experience 如何提供給 Planner、Retry Planner 與 Rerollout 局部修復流程。

3.3 動態局部修復機制

3.3.1 局部修復觸發條件

3.3.2 錯誤資訊與相依資料蒐集

3.3.3 修復 API 搜尋

3.3.4 局部計畫修復策略

本研究採用插入步驟（Insert）、替換步驟（Replace）與刪除步驟（Delete）三種局部計畫修復策略。

3.3.5 局部修復與重新規劃的切換條件

局部錯誤由 Rerollout 處理；當錯誤涉及整體任務理解或計畫結構時，系統則回退至 Retry Planner 重新規劃。

3.4 CBR-APIKGAgent 整合運作流程

本節整合原子化經驗、Rerollout 與重新規劃機制，說明 CBR-APIKGAgent 的完整決策流程。

第四章 系統實作

4.1 開發環境與工具

4.1.1 LangGraph

4.1.2 Neo4j

4.1.3 大型語言模型與嵌入模型

4.1.4 其他開發環境

4.2 APIKGAgent 基礎實作與修改位置

4.2.1 主控制流程

4.2.2 API 搜尋流程

4.2.3 任務執行與重新規劃流程

4.2.4 CBR-APIKGAgent 修改位置

本節說明經驗檢索與提示注入位置、Rerollout 插入位置，以及原始流程與修改後流程的程式差異。

4.3 原子化經驗建立與檢索實作

4.3.1 訓練資料成功任務軌跡萃取

4.3.2 原子化經驗資料格式

4.3.3 經驗向量化與語意檢索

4.3.4 Planner 經驗注入

4.3.5 Retry Planner 經驗注入

4.3.6 Rerollout 經驗注入

4.4 Rerollout 動態局部修復實作

4.4.1 修復觸發與錯誤證據整理

4.4.2 相依步驟資料取得

4.4.3 修復 API 搜尋流程

4.4.4 插入、替換與刪除策略實作

4.4.5 局部修復失敗與重新規劃回退

第五章 實驗與結果分析

5.1 研究問題

5.1.1 研究問題一

相較於原 APIKGAgent，CBR-APIKGAgent 對 AppWorld test_normal 任務完成率有何影響？

5.1.2 研究問題二

原子化經驗與動態局部修復分別對任務完成率及錯誤修復表現有何影響？

5.1.3 研究問題三

動態局部修復適用於哪些錯誤類型，其失敗情境與能力限制為何？

5.2 實驗設置

5.2.1 AppWorld 資料集與任務範圍

5.2.2 評估指標

5.2.3 實驗環境與模型設定

本研究於 AppWorld [3] 環境中進行實驗，並使用其任務資料與狀態式評估機制驗證代理人的任務完成情形。

5.2.4 比較方法

5.3 整體效能比較

本節以 APIKGAgent 與 CBR-APIKGAgent 為主要比較對象；其他方法僅於資料集、實驗設定與評估方式一致時進行直接比較。

5.4 消融實驗

5.4.1 實驗組別

本研究分別比較基礎系統、僅使用原子化經驗、僅使用動態局部修復，以及同時使用兩項機制的完整系統。



5.4.2 原子化經驗之影響

5.4.3 動態局部修復之影響

5.4.4 兩項機制之綜合影響

5.5 動態局部修復分析

5.5.1 修復觸發與執行情形

5.5.2 成功修復案例

5.5.3 修復失敗案例

5.5.4 錯誤類型分析

5.5.5 修復策略分析

5.6 討論

5.6.1 原子化經驗的影響與適用情境

5.6.2 動態局部修復的影響與適用情境

5.6.3 系統限制與能力邊界

第六章 結論與未來研究

6.1 結論

6.1.1 研究成果總結

6.1.2 研究貢獻

6.1.3 研究限制

6.2 未來研究

6.2.1 修復經驗累積機制

6.2.2 多輪局部修復策略

6.2.3 約束感知修復機制

6.2.4 跨基準資料集驗證

參考文獻

- [1] L. Wang, C. Ma, X. Feng, *et al.*, “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, 186345, Mar. 2024. DOI: 10.1007/s11704-024-40231-1.
- [2] 吳柏諭，一個由 LLM 驅動之 AI 代理人透過操作 API 自動完成任務之研究，碩士論文，國立臺北科技大學資訊工程系，2025。 <https://hdl.handle.net/11296/837n46>.
- [3] H. Trivedi, T. Khot, M. Hartmann, *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 16 022–16 076. DOI: 10.18653/v1/2024.acl-long.850.
- [4] J. S. Park, J. O’Brien, *et al.*, “Generative agents: Interactive simulacra of human behavior,” *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.
- [5] S. Yao, J. Zhao, D. Yu, *et al.*, “React: Synergizing reasoning and acting in language models,” *International Conference on Learning Representations (ICLR)*, 2023.
- [6] L. Wang, W. Xu, *et al.*, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [7] N. Shinn, F. Cassano, *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, 2023.
- [8] M. Li *et al.*, “Toolbench: Towards open tool learning for large language models,” *Advances in Neural Information Processing Systems*, 2023.
- [9] Y. Qin *et al.*, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” *Advances in Neural Information Processing Systems*, 2023.
- [10] A. Aamodt and E. Plaza, “Case-based reasoning: Foundational issues, methodological variations, and system approaches,” *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994. DOI: 10.3233/AIC-1994-7104.
- [11] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “ExpeL: LLM agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 19 632–19 642. DOI: 10.1609/aaai.v38i17.29936.
- [12] V. Karpukhin, B. Oguz, S. Min, *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

- [13] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [14] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, 2023.
- [15] Z. Chen *et al.*, “Divide-verify-refine: Self-correction via verification and refinement for complex tasks,” *arXiv preprint*, 2024.



附錄

6.3 Prompts

在此整理撰寫論文、設計實驗與系統實作時使用的 prompts。

6.4 程式碼

在此整理重要程式碼片段與關鍵實作說明。

